

PLAYHACK · SDE TRACK · IIT GUWAHATI

PLAYHACK

Campus sports facility booking where the winner of a race for one court is decided by Postgres, not by our code.

TEAM **INNOVAIT**

INNOVAIT-HACKATHON.VERCEL.APP

0 OVERLAPPING BOOKINGS, EVER

AT 6 PM, FIFTY STUDENTS WANT THE SAME COURT

"The system must confirm **exactly one** and reject the rest, without corrupting data."

Almost every booking system answers this the same way — and almost every one of them is wrong.

```
// the question
const taken = await isSlotTaken(court, slot);

// the answer
if (!taken) await insertBooking(court, slot);
```

Both lines are individually correct. Between them is a **gap** — and under load another request commits inside it.

THE WRITE *IS* THE DECISION

```
ALTER TABLE bookings ADD CONSTRAINT bookings_no_overlap  
    EXCLUDE USING gist (facility_id WITH =, during WITH &&)  
    WHERE (status = 'confirmed');
```

There is no question and no answer. A booking carries a time range, and the database **physically cannot store two overlapping ranges** for the same facility. The loser is rejected with SQLSTATE **23P01** before it ever becomes a row — no availability check on the write path, and no application lock on the slot.

PARTIAL OVERLAP IS THE CASE THAT HAPPENS

	UNIQUE (FACILITY, STARTS_AT)	EXCLUDE ... WITH &&
Two bookings at 18:00	rejected	rejected
18:00–19:00 vs 18:30–19:30	both stored	rejected
A two-hour closure over four slots	not expressible	one row
Cancelling frees the slot	needs a delete	status flip

A booking from 18:30 does not share a start time with one from 18:00 — but it **overlaps** it, and overlap is what matters to a student holding a racket.

THE GAP BETWEEN THE QUESTION AND THE ANSWER

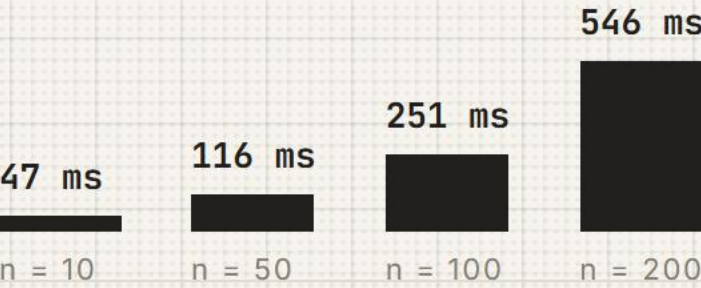
Fifty students tap Book on the same court at 18:00. The same burst, fired twice at the deployed app — once at a textbook availability check, once at a database that refuses to store the overlap.

1225

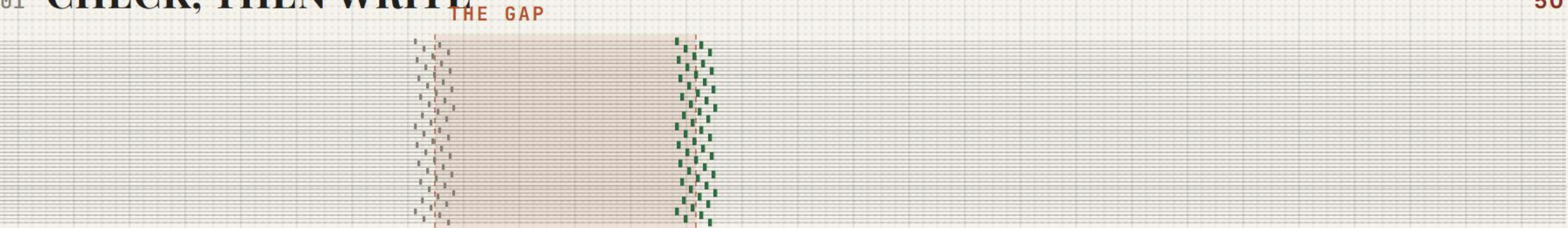
overlapping pairs left behind by **check-then-write**, swept across the whole table

0

from the identical burst, once the constraint is carrying the decision



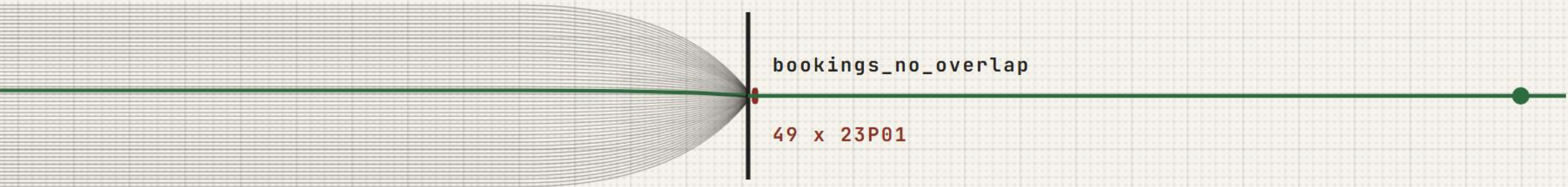
01 CHECK, THEN WRITE



Every request reads “free”, then writes. The shaded interval is the gap between the question and the answer — and fifty answers commit inside it.

50 CONFIRMED · NOTHING ERRORED

02 THE WRITE IS THE DECISION



No read. Fifty inserts race one exclusion constraint; Postgres rejects forty-nine with **SQLSTATE 23P01** before they can become rows.

1 CONFIRMED · 49 REJECTED

```
ALTER TABLE bookings
ADD CONSTRAINT bookings_no_overlap
EXCLUDE USING gist (
  facility_id WITH =,
  during WITH &&
) WHERE (status = 'confirmed');
```

Measured on the deployed app — Vercel **sin1**, Neon Postgres **ap-southeast-1**, warm, median of three. Server time is **sub-linear in n**: twenty times the load for ten times the time. The overlap count is a whole-table sweep after every run, not a check of the slot just tested.

THE SAME BURST, FIRED TWICE AT PRODUCTION

FIFTY REQUESTS. ONE COURT.

CHECK, THEN WRITE

50 bookings confirmed. Nothing errored.

1225 overlapping pairs, swept across the whole table

Fifty students walk to the same court.

THE WRITE IS THE DECISION

1 confirmed — 49 rejected with 23P01

0 overlapping pairs, across 380+ confirmed rows

Every rejection is typed and carries three alternative slots.

MEASURED ON THE DEPLOYED APP

SUB-LINEAR, NOT LINEAR

All contenders attempt the insert at once. One wins; the rest block on its uncommitted row and are released together the moment it commits. **Nothing in the write path serialises on n.**

Vercel **sin1**, Neon **ap-southeast-1**. Median of three warm runs.

MODE	REQUESTS	SERVER	CONFIRMED	OVERLAPS
naive	50	37 ms	50	1225
safe	10	47 ms	1	0
safe	50	116 ms	1	0
safe	100	251 ms	1	0
safe	200	546 ms	1	0

CORRECTNESS IS THE FLOOR

Idempotency keys. A retried submit returns the original booking. A double-tap on hostel wifi is the common case, not the edge case.

Typed rejections. Losing returns `SLOT_TAKEN`, `QUOTA_EXCEEDED`, `OVERLAPS_OWN` — with three alternative slots and a queue position.

Waitlist promotion. Cancelling frees the slot *and* offers it to the next student in the same transaction — no window where it is free but unowned.

Maintenance closures. A closure is a booking row. One invariant, two features: a manager cannot close a court out from under a confirmed reservation.

Fair draw. Peak slots go to a seeded, weighted, published lottery. First-come-first-served rewards the best wifi, not the keenest player.

THE PRODUCT

BUILT FOR A STUDENT ON A PHONE AT 5:58 PM

Indian Institute of Technology Guwahati · Students' Sports Board

12:14 IST

PLAYHACK

CAMPUS FACILITY BOOKING

BOOKMY BOOKINGSRACEFAIR DRAWINSIGHTSOPS

D

REACTING AS Demo Student

SWITCH BEADER

← BACK TO THE GRID

TENNIS

TENNIS COURT A

SPORTS COMPLEX, NEAR LEHTT

Floodlit hard court. Evening slots go fast.

HOURS

06:00-21:00

SLOT

60 min

CAPACITY

4

PLAYING SURFACE, PARKINGS TO SCALE

01

SUN 30 AUG

13 OF 15 OPEN

A slot is yours only once the confirmation appears. Nothing is held by clicking.

TODAY

TOMORROW

MON 31 AUG

TUE 1 SEPT

WED 2 SEPT

THU 3 SEPT

FRI 4 SEPT

SAT 5 SEPT

OPEN

YOURS

TAKEN

CLOSED

LIVE

06:00 OPEN	07:00 OPEN	08:00 OPEN	09:00 OPEN
10:00 OPEN	11:00 OPEN	12:00 OPEN	13:00 OPEN
14:00 PRISHA GOGOI	15:00 OPEN	16:00 OPEN	17:00 PEAK
18:00	19:00 PEAK	20:00 PEAK	

Booking.

The day as a fixture table, live over SSE.

Indian Institute of Technology Guwahati · Students' Sports Board

12:14 IST

PLAYHACK

CAMPUS FACILITY BOOKING

BOOKMY BOOKINGSRACEFAIR DRAWINSIGHTSOPS

D

REACTING AS Demo Student

SWITCH BEADER

INNOVATION · FAIR ALLOCATION

WINNING A RACE IS NOT DESERVING THE COURT

The exclusion constraint guarantees that exactly one booking survives a stampede. It does not decide who should win — and first-come-first-served decides that badly. At six the winner is whoever has the best wifi, the newest phone, or a script.

So peak slots are not released as a race. Requests arriving inside a short window become entries, and one weighted, seeded draw picks the winner. Fairness sits on top of correctness: the winner's booking still goes through the same constrained path as any other.

01 HOW THE DRAW WORKS

A draw a student cannot explain to the person who lost is not fair, only opaque.

01 A WINDOW, NOT A STARTING PISTOL

Every request that arrives inside the window becomes an entry. Arriving in the first millisecond buys nothing, so there is no advantage in a faster phone, a closer router, or a script left running.

02 WEIGHT BY RELIABILITY

Each entry carries weight 50 + reliability/2. A student who turns up keeps a full ticket; a record of no-shows costs some of it. The floor of 50 means nobody is ever locked out.

03 SEEDED, AND PUBLISHED

The winner is sha256(seed || entries) over the entrant list sorted by id. The seed is published with the result, so anyone can recompute the winner and check it.

02 RUN A DRAW

Opening a window clears the slot first, so every draw starts from the same state.

Fair draw.

Rules and seed published before the draw.

Indian Institute of Technology Guwahati · Students' Sports Board

12:14 IST

PLAYHACK

CAMPUS FACILITY BOOKING

BOOKMY BOOKINGSRACEFAIR DRAWINSIGHTSOPS

D

REACTING AS Demo Student

SWITCH BEADER

OPERATIONS · INSIGHTS

WHERE THE CAMPUS ACTUALLY PLAYS

Every figure on this page aggregates the same bookings table the booking path writes to. There is no reporting copy, no nightly export and nothing that can drift out of step with what actually happened.

UPCOMING BOOKINGS

152

BOOKED LAST 7 DAYS

423

ACTIVE STUDENTS

199

ON WAITLISTS

12

NO-SHOW RATE

13%

01 DEMAND BY HOUR

Bookings started in each hour, trailing 14 days. Solid bars are the peak band.

02 UTILISATION BY FACILITY AND HOUR

Denser hatching means busier. An empty band at the right of a row is capacity nobody is using.

Insights.

Aggregated from the booking table itself.

VERIFIED, NOT ASSERTED

EVERY CLAIM ON THESE SLIDES IS REPRODUCIBLE

11/11

browser journey against
production — book, confirm,
cancel, slot returns

19

unit tests, run in CI against a
real Postgres 17

0

npm audit vulnerabilities

```
npm run test      # concurrency + lottery
npm run invariant # whole-table sweep
npm run e2e       # 11 browser checks
```

The overlap count is never a check of the slot just tested — it is **every confirmed row against every other row** on the same court, run after every race.

WHAT IS DELIBERATELY OUT OF SCOPE

No passwords. Identity is a signed cookie over a seeded roster, with a “switch reader” control — so a judge can race two students without two browser profiles. Swapping in SSO replaces one function.

The demo endpoints are unauthenticated and rate limited rather than gated, because the race demo has to stay one click away.

What is enforced: ownership scoped inside every UPDATE, zod on every request body, bind parameters on every query, no `sql.unsafe` anywhere.

Next: institute SSO, push notifications for waitlist offers, and a real limiter keyed on identity rather than on a header the client controls.

ONE PLATFORM. ZERO CLASHES.

FIFTY STUDENTS. ONE COURT. ONE WINNER.

Run the race yourself — the burst is fired at a real Postgres table through a real HTTP endpoint, and the whole table is swept afterwards.

innovait-hackathon.vercel.app/race

TEAM INNOVAIT · PLAYHACK SDE TRACK